



Palindrome pattern matching

Tomohiro I^{*}, Shunsuke Inenaga, Masayuki Takeda

Department of Informatics, Kyushu University, Japan

ARTICLE INFO

Keywords:

Palindromes
Pattern matching
Linear time algorithms
Border arrays
Suffix trees

ABSTRACT

A palindrome is a string that reads the same forward and backward. For a string x , let $Pals(x)$ be the set of all maximal palindromes of x , where each maximal palindrome in $Pals(x)$ is encoded by a pair (c, r) of its center c and its radius r . Given a text t of length n and a pattern p of length m , the palindrome pattern matching problem is to compute all positions i of t such that $Pals(p) = Pals(t[i : i + m - 1])$. We present linear-time algorithms to solve this problem.

© 2012 Elsevier B.V. All rights reserved.

1. Introduction

A palindrome is a symmetric string that reads the same forward and backward. Namely, a string w is a palindrome if $w = xax^R$ where x is a string, x^R is a reversal of x , and a is either a single character or the empty string.

Recently, palindromic structures in strings have been extensively studied: a string of length n is called *palindromic rich* (or simply *rich*) if it contains $n + 1$ distinct palindromes (including the empty string). It is known that any string of length n can contain at most $n + 1$ distinct palindromes [1]. A unified study of palindromic richness of finite and infinite strings was initiated in [2]. A close relationship between palindromic richness and the Burrows–Wheeler transform [3] was recently discovered in [4]. Another concept regarding palindromic structures is *palindrome complexity* [5–7] of a string, which is the number of palindromic substrings of a given length in the string.

There exist several efficient algorithms that solve interesting problems on palindromes: a linear-time algorithm to check if a given string is palindromic rich or not, is presented in [8]. One can compute the set of all maximal palindromes of a given string in linear time [9]. The reverse engineering problem of computing a string from a given set of maximal palindromes is solvable in linear time [10], and the reverse problem from another type of palindromes (to be denoted by $Lpal$ in this paper) is also considered in [11].

In this paper, we introduce a new paradigm of pattern matching based on palindromes in strings. Two strings of same length m are said to be *pal-equivalent* iff the length of the maximal palindrome at every center in the strings is equal [10]. Given a text string t and a pattern string p , we are interested in finding all text positions i ($1 \leq i \leq n$) such that p and $t[i : i + m - 1]$ are pal-equivalent, where n and m are text and pattern lengths, respectively. This problem is called the *palindrome pattern matching*.

It is not difficult to see that the palindrome pattern matching problem can be solved in $O(nm)$ time: we pre-compute all maximal palindromes for t and p using linear time algorithms [9,12]. For every text position i , we compare the length of the maximal palindromes of t at position $i + j - 1$ and that of p at position j for every $1 \leq j \leq m$. If a maximal palindrome of the text “goes over” the interval $[i : i + j - 1]$, then the left and right arms of the maximal palindrome are trimmed accordingly for comparison.

For small alphabets, we can solve the problem in linear time by reducing the problem to the parameterized pattern matching problem. Baker [13] introduced the notion of parameterized match, where two strings are said to parameterized match iff there exists a bijection on the alphabet which transforms one string to the other. She also proposed an

^{*} Corresponding author. Tel.: +81 92 802 3786.

E-mail addresses: tomohiro.i@inf.kyushu-u.ac.jp (T. I.), inenaga@inf.kyushu-u.ac.jp (S. Inenaga), takeda@inf.kyushu-u.ac.jp (M. Takeda).

$O(n + m)$ -time algorithm to compute all substrings locations of a text t where a pattern p parameterized matches. Later in [10], it was shown that if the alphabet size is at most 3, then two strings are pal-equivalent iff those strings parameterized match. Hence the palindrome pattern matching can be solved in $O(n + m)$ time for ternary and smaller alphabets.

In this paper, we present efficient solutions for larger alphabets. Firstly, we present an algorithm which solves the problem in $O(n + m)$ time for *arbitrary* alphabets. This algorithm is a palindrome-pattern-matching version of the Morris–Pratt [14] pattern matching algorithm. Secondly, we propose another algorithm that uses a new text indexing structure called the *palindrome suffix trees*. We show that palindrome suffix trees can be constructed in $O(n \cdot \min\{\sqrt{\log n}, \frac{\log \sigma}{\log \log \sigma}\})$ time with $O(n)$ space, where σ is the alphabet size. Using the palindrome suffix tree, we can solve the problem in $O(m \cdot \min\{\sqrt{\log n}, \frac{\log \sigma}{\log \log \sigma}\} + r)$ time, where r is the number of text positions to report. Obviously our palindrome suffix tree approach provides us with a linear time solution when σ is a constant.

The algorithms of this paper are applicable to several practical problems, e.g., in bioinformatics. For instance, similar palindromic sequences often need to be identified in DNA and RNA sequence analysis [12]. Sequences having similar palindromic structures may code for similar 3-D structures of the respective molecules, leading to possible functional interpretation of the identified sequences. Due to the size of genomes, efficiency of search methods is of great importance.

2. Preliminaries

Let Σ be a finite *alphabet*. An element of Σ^* is called a *string*. The length of a string w is denoted by $|w|$. The empty string ε is a string of length 0, that is, $|\varepsilon| = 0$. Let $\Sigma^+ = \Sigma^* - \{\varepsilon\}$. When a string w is represented by the concatenation of strings x , y and z (i.e., $w = xyz$), then x , y and z are called a *prefix*, *substring*, and *suffix* of w , respectively. The i -th character of a string w is denoted by $w[i]$ for $1 \leq i \leq |w|$, and the substring of a string w that begins at position i and ends at position j is denoted by $w[i : j]$ for $1 \leq i \leq j \leq |w|$, i.e., $w[i : j] = w[i]w[i+1] \dots w[j]$. For convenience, let $w[i : j] = \varepsilon$ if $j < i$.

For any string w , let w^R denote the reversed string of w , that is, $w^R = w[|w|] \dots w[2]w[1]$. A string w is called a *palindrome* if $w = w^R$. If $|w|$ is even, then w is called an *even palindrome*, that is, $w = xx^R$ for some $x \in \Sigma^*$. If $|w|$ is odd, then w is called an *odd palindrome*, that is, $w = xax^R$ for some $x \in \Sigma^*$ and $a \in \Sigma$. The *radius* of a palindrome w is $\frac{|w|}{2}$.

The *center* of a palindromic substring $w[i : j]$ of a string w is $\frac{i+j}{2}$. A palindromic substring $w[i : j]$ is called the *maximal palindrome* at the center $\frac{i+j}{2}$ if no other palindromes at the center $\frac{i+j}{2}$ have a larger radius than $w[i : j]$, i.e., if $w[i-1] \neq w[j+1]$, $i = 1$, or $j = |w|$. In particular, a maximal palindrome $w[i : |w|]$ is called a *suffix palindrome* of w .

Let $Pals(w)$ be the set of all center-distinct maximal palindromes where each element is encoded by a pair of its center and radius, namely,

$$Pals(w) = \left\{ (c, r) \mid \begin{array}{l} w[c - r + 0.5 : c + r - 0.5] \text{ is a maximal palindrome} \\ \text{at center } c = 1, 1.5, 2, \dots, n \end{array} \right\}.$$

Also, let

$$SPals(w) = \{(c, r) \mid (c, r) \in Pals(w), c + r - 0.5 = |w|\},$$

namely, $SPals(w)$ represents the set of all suffix palindromes of w .

For example, let $w = \text{abbacabbba}$. Then

$$\begin{aligned} Pals(w) &= \{(1, 0.5), (1.5, 0), (2, 0.5), (2.5, 2), (3, 0.5), (3.5, 0), (4, 0.5), (4.5, 0), \\ &\quad (5, 3.5), (5.5, 0), (6, 0.5), (6.5, 0), (7, 0.5), (7.5, 1), (8, 2.5), (8.5, 1), \\ &\quad (9, 0.5), (9.5, 0), (10, 0.5)\} \text{ and} \\ SPals(w) &= \{(8, 2.5), (10, 0.5)\}. \end{aligned}$$

Theorem 1 ([9]). *For any string w of length m , $Pals(w)$ can be computed in $O(m)$ time.*

Throughout this paper, we assume that the elements of $Pals(w)$ are sorted in increasing order of centers c . Actually, the algorithm of [9] computes the elements of $Pals(w)$ in this order.

In this paper, we tackle the following problem.

Problem 1 (*Palindrome Pattern Matching, Pal-matching in Short*). Given a text string t of length n and a pattern string p of length m , compute all positions i of t such that $Pals(p) = Pals(t[i : i + m - 1])$.

3. Linear-time palindrome pattern matching algorithm

To achieve a linear time solution to Problem 1, we design a pal-matching version of the Morris–Pratt algorithm [14].

Definition 1. A *palindrome matching border* (*pal-border* for short) of a string p of length m is any integer j s.t. $0 \leq j < m$ and $Pals(p[1 : j]) = Pals(p[m - j + 1 : m])$.

For example, the set of pal-borders of string $p = \text{aabcbdaacdbcc}$, is $\{7, 2, 1, 0\}$, since $\text{Pals}(\text{aabcbdaa}) = \text{Pals}(\text{aacdbcc})$, $\text{Pals}(\text{aa}) = \text{Pals}(\text{cc})$, $\text{Pals}(\text{a}) = \text{Pals}(\text{c})$, and $\text{Pals}(\varepsilon) = \text{Pals}(\varepsilon)$.

Let $\mathcal{N}$ be the set of non-negative integers. For any string p of length m , let $\text{Pal_Border}_p : \mathcal{N} \rightarrow \mathcal{N}$ be the function such that $\text{Pal_Border}_p(m)$ equals the largest pal-border of string p . When clear from the context, we abbreviate Pal_Border_p as Pal_Border . Since $\text{Pal_Border}(m)$ is strictly smaller than m , we finally obtain 0 by iteratively applying the function Pal_Border to m . For any function $f : \mathcal{N} \rightarrow \mathcal{N}$ and any $m, k \in \mathcal{N}$, we define $f^k(m)$ as follows: $f^k(m) = f(m)$ if $k = 1$, and $f^k(m) = f(f^{k-1}(m))$ if $k \geq 2$. Similar to a standard border of a string [14], the following lemma holds.

Lemma 1. For any string p of length m , let k be the smallest integer such that $\text{Pal_Border}^k(m) = 0$. Then

$$\text{Pal_Border}(m), \text{Pal_Border}^2(m), \dots, \text{Pal_Border}^k(m)$$

are all the pal-borders of p with $m > \text{Pal_Border}(m) > \text{Pal_Border}^2(m) > \dots > \text{Pal_Border}^k(m) = 0$.

Definition 2. The *palindrome border array* (pal-border array) β_p of a string p of length m is an integer array of length m such that $\beta_p[i] = \text{Pal_Border}_{p[1:i]}(i)$ for each $1 \leq i \leq m$.

For example, for string $p = \text{aabbbaa}$, we have $\beta_p = [0, 1, 1, 2, 3, 4]$. When it is clear from the context, we abbreviate β_p as β .

In what follows, we present how to compute the pal-border array β_p of a given string p in linear time.

For any string w of length $m \geq 1$, let Lpal_w be an integer array of length m such that

$$\text{Lpal}_w[i] = \max\{i - k + 1 \mid w[k : i] = w[k : i]^R, 1 \leq k \leq i\}.$$

That is, the value of $\text{Lpal}_w[i]$ is equal to the length of the longest palindrome that ends at position i in w , for every $1 \leq i \leq m$.¹ Note that the above palindrome $w[k : i]$ is not necessarily a maximal palindrome at center $\frac{k+i}{2}$ in w .

For example, for string $w = \text{abbacabbba}$, $\text{Lpal}_w = 1 \ 1 \ 2 \ 4 \ 1 \ 3 \ 5 \ 7 \ 3 \ 5$.

The following lemma is a key to solve Problem 1 of pal-matching.

Lemma 2. For any strings $w, z \in \Sigma^+$, $\text{Pals}(w) = \text{Pals}(z)$ iff $\text{Lpal}_w = \text{Lpal}_z$.

Proof. ($\implies$) We prove the claim by contradiction. Assume for contrary that $\text{Lpal}_w \neq \text{Lpal}_z$. Then there exists position i such that $\text{Lpal}_w[i] \neq \text{Lpal}_z[i]$. Assume w.l.o.g. that $\text{Lpal}_w[i] < \text{Lpal}_z[i]$. Let $k = (\text{Lpal}_z[i])/2$. The radius of the maximal palindrome centered at position $i - k + 0.5$ of w is less than k , however, that of the maximal palindrome centered at position $i - k + 0.5$ of z is at least k . This contradicts the assumption that $\text{Pals}(w) = \text{Pals}(z)$. Hence if $\text{Pals}(w) = \text{Pals}(z)$, then $\text{Lpal}_w = \text{Lpal}_z$.

($\impliedby$) We prove the claim by contradiction and infinite descent. Assume for contrary that $\text{Pals}(w) \neq \text{Pals}(z)$. Then there exists center c such that $(c, r) \in \text{Pals}(w)$, $(c, u) \in \text{Pals}(z)$, and $r \neq u$. Assume w.l.o.g. that $r < u$.

In what follows, we consider position $j = \lceil c + u \rceil - 1$.

1. When $\text{Lpal}_w[j] < 2u$. Since $(c, u) \in \text{Pals}(z)$, $\text{Lpal}_z[j] \geq 2u$. This contradicts the assumption that $\text{Lpal}_w = \text{Lpal}_z$.
2. When $\text{Lpal}_w[j] \geq 2u$. Let $k = (\text{Lpal}_w[j])/2$. Then clearly w has a palindrome that is centered at $j - k + 0.5$ and is of radius k . Also z has a palindrome that is centered at $j - k + 0.5$ and is of radius k , since otherwise it contradicts the assumption that $\text{Lpal}_w = \text{Lpal}_z$. Then there exists center $c' < c$ such that $(c', r) \in \text{Pals}(w)$, $(c', u) \in \text{Pals}(z)$, and $r < u$. (See also Fig. 1.)

The same must hold for those smaller centers, ad infinitum. However, this is impossible since w and z are finite strings.

Hence if $\text{Lpal}_w = \text{Lpal}_z$, then $\text{Pals}(w) = \text{Pals}(z)$. $\square$

It is shown in [8] that Lpal_w can be computed in linear time from $\text{Pals}(w)$. The following lemma is essentially the same as what is claimed in [8], but is more specifically tailored for our needs.

Lemma 3. Let w be any string of length m . Given $\text{Pals}(w)$, Lpal_w can be computed in $O(m)$ time, in an on-line fashion, from $\text{Lpal}_w[1]$ to $\text{Lpal}_w[m]$.

Proof. For any position i of w with $1 \leq i \leq m$, the value of $\text{Lpal}_w[i]$ is equal to $2(i - c) + 1$ where c is the smallest center of a maximal palindrome $(c, r) \in \text{Pals}(w)$ such that $c + r \geq i$. Hence we process the given string w from left to right.

Assume that we have computed $\text{Lpal}_w[1 : i]$ and let $(c, r) \in \text{Pals}(w)$ with $\text{Lpal}_w[i] = 2(i - c) + 1$. Now we compute $\text{Lpal}_w[i + 1]$. If $c + r \geq i + 1$, then $\text{Lpal}_w[i + 1] = 2((i + 1) - c) + 1$. Otherwise, we increment the value of c by 0.5 until satisfying $c + r \geq i + 1$, where r is the radius of the maximal palindrome with the updated center c .

A pseudo-code of the algorithm is shown in Algorithm 1. The correctness should be clear from the above arguments. Note that the value of c does not decrease and does not exceed the value of i . Also, (c, r) can be picked up from $\text{Pals}(w)$ in constant time at each step, since $\text{Pals}(w)$ is sorted in increasing order of c . Consequently the time complexity is linear in m . $\square$

Let w be any string of length m , and let s and i be any integers with $1 \leq s \leq i \leq m$. Here we consider computing $\text{Lpal}_{w[s:i]}[i - s + 1]$ from $\text{Pals}(w)$. By the definition of Lpal , the value of $\text{Lpal}_{w[s:i]}[i - s + 1]$ is equal to $2(i - c) + 1$, where (c, r) is the maximal palindrome in $\text{Pals}(w)$ such that c is the smallest center satisfying $c \geq (s + i)/2$ and $c + r \geq i$ (See also Fig. 2). We call this center c the *active center* for s and i w.r.t. w , and denote it by $\text{AC}_w(s, i)$. It holds that $\text{Lpal}_{w[s:i]}[i - s + 1] = 2(i - \text{AC}_w(s, i)) + 1$.

¹ The notion of $\text{Lpal}_w[i]$ was previously introduced in [8], denoted $\text{LPS}[i]$ therein.

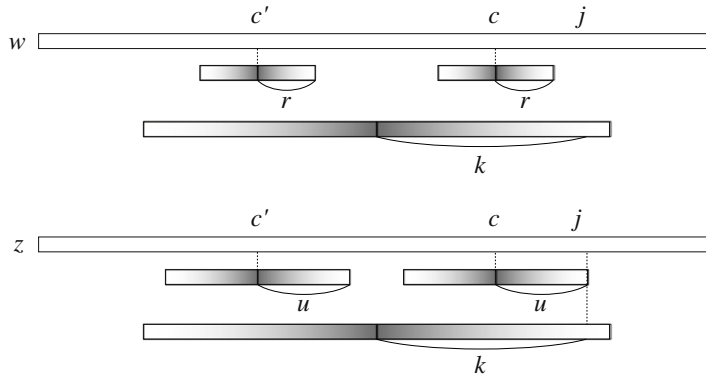


Fig. 1. Illustration for infinite descent in the proof of Lemma 2.

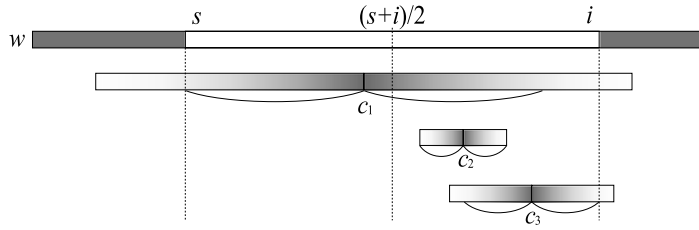


Fig. 2. If (c_3, r_3) is the maximal palindrome in $Pals(w)$ such that c_3 is the smallest center satisfying $c_3 \geq (s+i)/2$ and $c_3 + r_3 \geq i$, c_3 is the active center for s and i , and $Lpal_{w[s:i]}[i-s+1] = 2(i-c_3) + 1$. Note that c_1 is not the active center for s and i since $c_1 < (s+i)/2$.

Algorithm 1: On-line algorithm to compute $Lpal_w$ from $Pals(w)$.

Input: $Pals(w)$ of a string w of length m .

Output: $Lpal_w[1 : m]$.

```

1  $c \leftarrow 1$ ; let  $(c, r) \in Pals(w)$ ;
2 for  $i \leftarrow 1$  to  $m$  do
3   while  $c + r < i$  do
4      $c \leftarrow c + 0.5$ ; let  $(c, r) \in Pals(w)$ ;
5    $Lpal_w[i] \leftarrow 2(i - c) + 1$ ;
6 return  $Lpal_w[1 : m]$ ;
```

Lemma 4. Let w be any string of length m . For any integers s, i, s', i' with $1 \leq s \leq i \leq m$ and $1 \leq s' \leq i' \leq m$, if $s \leq s'$ and $i \leq i'$, then $AC_w(s, i) \leq AC_w(s', i')$.

Proof. Assume for contrary that $AC_w(s, i) > AC_w(s', i')$. Since $AC_w(s, i) \leq i$, $AC_w(s', i') < i$. Let $(AC_w(s', i'), r) \in Pals(w)$. It follows from $AC_w(s', i') \geq (s' + i')/2 \geq (s + i)/2$ and $AC_w(s', i') + r \geq i' \geq i$ that $AC_w(s', i') \geq (s + i)/2$ and $AC_w(s', i') + r \geq i$. However this contradicts that $AC_w(s, i)$ is the active center for s and i w.r.t. w . $\square$

In the algorithms which follow, we will need to know the value of $Lpal_{w[s:i]}[i-s+1]$ for some s and i . It seems difficult to compute $Lpal_{w[s:i]}[i-s+1]$ in constant time for “randomly” chosen s and i , with $O(m)$ -time preprocessing. Nevertheless, Lemma 4 suggests that, if s and i monotonically increase from 1 to m , then the total cost for computing $Lpal_{w[s:i]}[i-s+1]$ for all s and i never exceeds the number of the centers in w , which is $2m - 1$. The point is that all the following algorithms only require to compute $Lpal_{w[s:i]}[i-s+1]$ for monotonically increasing positions s and i , with $1 \leq s \leq i \leq m$.

Lemma 5. For any string p of length m , β_p can be computed in $O(m)$ time.

Proof. Algorithm 2 describes our algorithm. This algorithm is mostly the same as the linear-time algorithm for computing a standard border array of a string [14], except that we match the values of $Lpal$ instead of characters.

We firstly compute $Pals(p)$ and $Lpal_p[1 : m]$. This takes $O(m)$ time by Theorem 1 and Lemma 3. Then we compute $\beta_p[1 : m]$ in ascending order. Consider the i -th iteration of the **for** loop of Line 4. Here we have computed $\beta_p[1 : i - 1]$, and variable j is set to be $\beta_p[i - 1]$. Next we compute $Lpal_{p[i-j:i]}[j + 1]$ by shifting the current center c right to $AC_p(i - j, i)$. If $Lpal_p[j + 1] = Lpal_{p[i-j:i]}[j + 1]$, $\beta_p[i] = j + 1$. Otherwise, we set j to be $\beta_p[j]$ and check again if $Lpal_p[j + 1] = Lpal_{p[i-j:i]}[j + 1]$ or not. The above procedure is repeated until j , such that $Lpal_p[j + 1] = Lpal_{p[i-j:i]}[j + 1]$, is found. Note that we break this loop at the latest when $j = 0$, since $Lpal_p[1] = Lpal_{p[i:i]}[1] = 1$.

Algorithm 2: Algorithm to compute β_p of a given string p .

Input: String p of length m .
Output: Pal-border array $\beta_p[1 : m]$.

```

1 compute  $Pals(p)$  and  $Lpal_p[1 : m]$ ;
2  $\beta_p[1] \leftarrow 0$ ;
3  $j \leftarrow 0$ ;  $c \leftarrow 0$ ;
4 for  $i \leftarrow 2$  to  $m$  do
5   while true do
6      $c \leftarrow \max\{c, i - j/2\}$ ; let  $(c, r) \in Pals(p)$ ;
7     while  $c + r < i$  do /* Shift  $c$  to  $AC_p(i - j, i)$ . */
8        $c \leftarrow c + 0.5$ ; let  $(c, r) \in Pals(p)$ ;
9       /*  $2(i - c) + 1 = Lpal_{p[i-j:i]}[j + 1]$ . */
10      if  $Lpal_p[j + 1] = 2(i - c) + 1$  then break;
11       $j \leftarrow \beta_p[j]$ ;
12     $j \leftarrow j + 1$ ;
13     $\beta_p[i] \leftarrow j$ ;
14 return  $\beta_p[1 : m]$ ;
```

Algorithm 3: Algorithm to solve pal-matching problem in linear time.

Input: Text string t of length n and pattern string p of length m .
Output: All positions i of t such that $t[i : i + m - 1]$ pal-matches p .

```

1 compute  $Pals(t)$ ,  $Lpal_p[1 : m]$ , and  $\beta_p[1 : m]$ ;
2  $j \leftarrow 0$ ;  $c \leftarrow 0$ ;
3 for  $i \leftarrow 1$  to  $n$  do
4   while true do
5      $c \leftarrow \max\{c, i - j/2\}$ ; let  $(c, r) \in Pals(t)$ ;
6     while  $c + r < i$  do /* Shift  $c$  to  $AC_t(i - j, i)$ . */
7        $c \leftarrow c + 0.5$ ; let  $(c, r) \in Pals(t)$ ;
8       /*  $2(i - c) + 1 = Lpal_{t[i-j:i]}[j + 1]$ . */
9       if  $Lpal_p[j + 1] = 2(i - c) + 1$  then break;
10       $j \leftarrow \beta_p[j]$ ;
11     $j \leftarrow j + 1$ ;
12    if  $j = m$  then
13       $j \leftarrow \beta_p[j]$ ; report  $i - m + 1$ ;
```

In each iteration of the **for** loop of Line 4, the value of j increases by at most 1. Since each execution of the **while** loop of Line 5 decreases the value of j at least 1 and $j \geq 0$, the **while** loop of Line 5 is executed at most m times in total. Moreover, since the value of c does not decrease and does not exceed the value of i , the total cost of the **while** loop of Line 7 is $O(m)$. Therefore Algorithm 2 runs in time linear in m . $\square$

Theorem 2. The pal-matching problem (Problem 1) can be solved in $O(n + m)$ time.

Proof. Algorithm 3 describes our algorithm. This algorithm is a pal-matching version of the Morris–Pratt algorithm [14].

We firstly compute $Pals(p)$ by Algorithm 1 and $Lpal_p[1 : m]$ by Algorithm 2 in $O(m)$ time, and $Pals(t)$ in $O(n)$ time. Consider the i -th iteration of the **for** loop of Line 3. Here variable j represents an integer such that $p[1 : j]$ and $t[i - j : i - 1]$ pal-match. Next we compute $Lpal_{t[i-j:i]}[j + 1]$ by shifting the current center c right to $AC_t(i - j, i)$. If $Lpal_p[j + 1] = Lpal_{t[i-j:i]}[j + 1]$, we break the **while** loop of Line 4. Otherwise, we set j to be $\beta_p[j]$ and check again if $Lpal_p[j + 1] = Lpal_{t[i-j:i]}[j + 1]$ or not. The above procedure is repeated until j , such that $Lpal_p[j + 1] = Lpal_{t[i-j:i]}[j + 1]$, is found. Note that we break this loop at the latest when $j = 0$, since $Lpal_p[1] = Lpal_{t[i:i]}[1] = 1$. After breaking the **while** loop of Line 4, we increment j by 1, and if j becomes m , the algorithm reports that $t[i - m + 1 : i]$ and $p[1 : m]$ pal-match.

In each iteration of the **for** loop of Line 3, the value of j increases by at most 1. Since each execution of the **while** loop of Line 4 decreases the value of j at least 1 and $j \geq 0$, the **while** loop of Line 4 is executed at most n times in total. Moreover, since the value of c does not decrease and does not exceed the value of i , the total cost of the **while** loop of Line 6 is $O(n)$. Therefore Algorithm 3 runs in $O(n + m)$ time. $\square$

	t	=	a	b	b	a	b	b	c	b	c
$Lpal_{t[1:9]}$	$[1 : 9]$	=	1	1	2	4	3	5	1	3	3
$Lpal_{t[2:9]}$	$[1 : 8]$	=		1	2	1	3	5	1	3	3
$Lpal_{t[3:9]}$	$[1 : 7]$	=			1	1	3	2	1	3	3
$Lpal_{t[4:9]}$	$[1 : 6]$	=				1	1	2	1	3	3
$Lpal_{t[5:9]}$	$[1 : 5]$	=					1	2	1	3	3
$Lpal_{t[6:9]}$	$[1 : 4]$	=						1	1	3	3
$Lpal_{t[7:9]}$	$[1 : 3]$	=							1	1	3
$Lpal_{t[8:9]}$	$[1 : 2]$	=								1	1
$Lpal_{t[9:9]}$	$[1 : 1]$	=									1

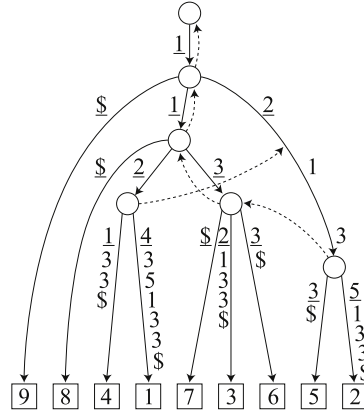


Fig. 3. Illustration of $Pal_ST(t)$ for string $t = abbabbcb$. The solid arrows represent the edges, and the broken arrows do the suffix links. The path from the root to each leaf s spells out $Lpal_{t[s:n]}[1 : s]\$$.

4. Palindrome suffix trees

In this section, we consider an indexing structure for pal-matching. We propose a new data structure called *palindrome suffix trees* (*pal-suffix trees* in short).

The pal-suffix tree of a string t , denoted $Pal_ST(t)$, is a compacted trie which represents $Lpal_{t[s:n]}[1 : n - s + 1]$ for all the suffixes $t[s : n]$ of t , where n is the length of t and $1 \leq s \leq n$. Each internal node of $Pal_ST(t)$ has at least two children, and the labels of two distinct out-going edges of each internal node must start with distinct non-negative integers. Moreover, for $Pal_ST(t)$ to have exactly n leaves, we use the following convention: each leaf of $Pal_ST(t)$ is uniquely labeled with integer s ($1 \leq s \leq n$) in such a way that the path from the root to leaf s spells out $Lpal_{t[s:n]}[1 : n - s + 1]\$$, where $\$$ is a special end-marker. The *length* of a node v , denoted $len(v)$, is the length of $Lpal$ represented by v . Fig. 3 illustrates $Pal_ST(abbabbcb)$.

4.1. Linear-space implementation of palindrome suffix trees

Notice that there are $O(n)$ distinct values for the elements of $Lpal_t[1 : n]$. For instance, consider $t = (ab)^{\frac{n}{2}}$. Then $Lpal_t[1 : n] = 1 \ 1 \ 3 \ 3 \ \dots \ n-1 \ n-1$. This suggests that an internal node of $Pal_ST(t)$ might have $O(n)$ children. However, the following lemma holds.

Lemma 6. For any string t , each node of $Pal_ST(t)$ has at most σ children, where σ is the number of distinct characters appearing in t .

Proof. For any string w , let $S(w) = SPals(w) \cup \{(|w| + 0.5, 0)\} - \{(|w|/2 + 0.5, |w|/2)\}$. To show the lemma, we consider the following claim.

Claim. Let w and z be any strings of length i s.t. $Pals(w) = Pals(z)$. For any integers j, k with $1 \leq j \leq i, 1 \leq k \leq i$ and $(\frac{i+j+1}{2}, \frac{i-j}{2}), (\frac{i+k+1}{2}, \frac{i-k}{2}) \in S(w)$, if $w[j] = w[k]$ then $z[j] = z[k]$.

Proof of Claim. When $j = k$, it is clear the claim holds. Then we consider the case $j \neq k$. Assume w.l.o.g. that $j < k$. Since $w[j+1 : i+j-k] = w^R[k+1 : i] = w[k+1 : i] = w^R[j+1 : i+j-k]$, $w[j+1 : i+j-k]$ is a palindrome. It follows from $w[j] = w[k]$ and $w[j+1 : i] = w^R[j+1 : i]$ that $w[j] = w[k] = w[i+j+1-k]$. Putting $w[j+1 : i+j-k] = w^R[j+1 : i+j-k]$ and $w[j] = w[i+j+1-k]$ together, we get $w[j : i+j+1-k]$ is a palindrome (See also Fig. 4). Since $Pals(w) = Pals(z)$, $z[j : i+j+1-k]$ and $z[j+1 : i]$ are palindromes, and thus $z[j] = z[i+j+1-k] = z[k]$. Hence the claim holds.

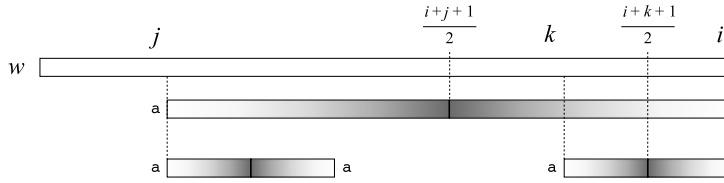


Fig. 4. Illustration for the proof of claim in Lemma 6.

Consider any substring w of length i of t . We introduce an equivalence relation on $S(w)$ such that

$$\left(\frac{i+j+1}{2}, \frac{i-j}{2}\right) \equiv \left(\frac{i+k+1}{2}, \frac{i-k}{2}\right) \iff w[j] = w[k],$$

where $1 \leq j \leq i$, $1 \leq k \leq i$, and $(\frac{i+j+1}{2}, \frac{i-j}{2}), (\frac{i+k+1}{2}, \frac{i-k}{2}) \in S(w)$. By definition, there are at most σ equivalence classes w.r.t. $\equiv$. Consider any substring z of t with $\text{Pals}(z) = \text{Pals}(w)$. Due to the above claim, the equivalence classes on $S(z)$ are identical to those on $S(w)$.

Let v be any node of $\text{Pal_ST}(t)$, and assume that the path from the root to v spells out Lpal_w . Note that every substring z of t that pal-matches w is represented by the same node v in $\text{Pal_ST}(t)$, since it has the same Lpal values as w , i.e., $\text{Lpal}_w = \text{Lpal}_z$. Therefore, the number of children of v is at most $d + 1$, where d is the number of equivalence classes on $S(w)$, which is bounded by σ . Hence the lemma holds. $\square$

In order to implement $\text{Pal_ST}(t)$ with $O(n)$ space, we encode the label of each edge as follows. Assume that there is an edge of $\text{Pal_ST}(t)$ labeled with x , where x is a sequence of positive integers. We encode x by a triple $(x[1], q, |x|)$, where $x[1]$ is the first element of x , q is a position of text t such that $x = \text{Lpal}_{t[s:n]}[q - s + 1 : q - s + |x|]$ for some $1 \leq s \leq n$, and $|x|$ is the length of the edge label. See Fig. 3 and focus on the edge which is labeled with 2 1 3. Choosing $s = 2$, the label is encoded by $(2, 3, 3)$ as $q = 3$, $|x| = 3$, and $\text{Lpal}_{t[2:9]}[2 : 4] = 2 \ 1 \ 3$. In Fig. 3, the first element of each edge label is shown underlined.

4.2. Palindrome pattern matching with palindrome suffix trees

Theorem 3. *Provided that $\text{Pal_ST}(t)$ and $\text{Pals}(t)$ are already computed, the pal-matching problem (Problem 1) can be solved in $O(mk + r)$ time, where k is the time cost to search at any branching node of $\text{Pal_ST}(t)$, and r is the output size.*

Proof. We compute Lpal_p using Algorithm 1 in $O(m)$ time. Then we search $\text{Pal_ST}(t)$ for $\text{Lpal}_p[1 : m]$. Assume that $\text{Lpal}_p[1 : j]$ matches the label of an out-going edge of the root node of $\text{Pal_ST}(t)$, with some $1 \leq j < m$. Assume the edge label is encoded as $(\text{Lpal}_{t[q:n]}[1], q, j)$, where $\text{Lpal}_{t[q:n]}[1 : j] = \text{Lpal}_p[1 : j]$. Let v be the node that represents $\text{Lpal}_{t[q:n]}[1 : j]$. Assume that there is an out-going edge of v , which is labeled with $(\text{Lpal}_{t[q'-j:n]}[j+1], q', j')$, where $\text{Lpal}_{t[q'-j:n]}[j+1] = \text{Lpal}_p[j+1]$ and $j' \geq 2$. This edge can be found in $O(k)$ time. Now we have to check whether $\text{Lpal}_{t[q'-j:n]}[j+2] = \text{Lpal}_p[j+2]$. Although q' is not necessarily equal to $q + j$, we can compute $\text{Lpal}_{t[q'-j:n]}[j+2]$ as follows: By the definition of $\text{Pal_ST}(t)$ it holds that $\text{Lpal}_{t[q'-j:n]}[1 : j+1] = \text{Lpal}_{t[q:n]}[1 : j+1]$, which implies that $\text{AC}_t(q' - j, q') = \text{AC}_t(q, q + j) + q' - (q + j)$. As described in Section 3, we can compute $\text{Lpal}_{t[q'-j:n]}[j+2]$ by shifting the current center from $\text{AC}_t(q' - j, q')$ to $\text{AC}_t(q' - j, q' + 1)$. Moreover, $\text{Lpal}_{t[q'-j:n]}[j+2] = \text{Lpal}_p[j+2]$ iff $\text{AC}_t(q' - j, q' + 1) - \text{AC}_t(q' - j, q') = \text{AC}_p(1, j+2) - \text{AC}_p(1, j+1)$. In light of this, the total cost for computing such values of Lpal is bounded by the cost for computing Lpal_p , which is $O(m)$. We continue the above procedure until either we find Lpal_p in $\text{Pal_ST}(t)$ or we find a mismatch. After Lpal_p is located in $\text{Pal_ST}(t)$, then we traverse the sub-tree rooted at the (possibly implicit) node that represents Lpal_p , and report the id of the leaves in the sub-tree, in $O(r)$ time. As k is the time cost to search each branching node of $\text{Pal_ST}(t)$, the total time cost is $O(mk + r)$. $\square$

Let us now focus on the term k in the above theorem. If we use a balanced binary search tree, we obtain $k = O(\log \sigma)$ by Lemma 6 and linear space implementation of $\text{Pal_ST}(t)$. Recall that the first element of every edge label of $\text{Pal_ST}(t)$ is an integer in range $[1, n]$, and the number of children of each node is at most σ . This allows us to use the following faster integer data structures:

- *q-fast tries* [15]: For a set S of d distinct integers on a bounded universe $U = \{1, 2, \dots, u\}$, this data structure permits membership queries and dynamic operations in $O(\sqrt{\log u})$ time using $O(d)$ space. In our context d is the number of children and $u = n$, and hence we obtain $k = O(\sqrt{\log n})$ with linear space implementation of $\text{Pal_ST}(t)$.
- *fusion trees* [16]: For a set S of d distinct integers on a bounded universe $U = \{1, 2, \dots, 2^b - 1\}$, where b is the machine word size, this data structure permits membership queries in $O(\frac{\log d}{\log \log d})$ time and dynamic operations in amortized $O(\frac{\log d}{\log \log d})$ time, using $O(d)$ space. In our context d is the number of children, which is at most σ . Hence whenever n is less than 2^b , we obtain $k = O(\frac{\log \sigma}{\log \log \sigma})$ with linear space implementation of $\text{Pal_ST}(t)$.

The former is a good choice in particular when σ is as large as $O(n)$ (e.g., integer alphabet), while the latter is more efficient than balanced binary search trees when n fits into the word size. Thus, assuming $n \leq 2^b - 1$, we obtain the following:

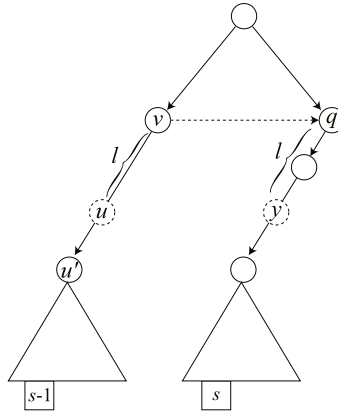


Fig. 5. Illustration of maintenance of the active point. u is the active point for $i - 1$, and y is a candidate for the active point for i .

Theorem 4. *Provided that $Pal_ST(t)$ and $Pals(t)$ are already computed, the pal-matching problem (Problem 1) can be solved in $O(m \cdot \min\{\sqrt{\log n}, \frac{\log \sigma}{\log \log \sigma}\} + r)$ time, where r is the output size.*

4.3. Constructing palindrome suffix trees

We employ Ukkonen's on-line construction techniques for suffix trees [17]. Here let us briefly review the behavior of the Ukkonen's algorithm. The algorithm processes the characters of a given string t of length n in ascending order. After processing the $(i - 1)$ -th character of t , the algorithm has constructed the suffix tree of $t[1 : i - 1]$. Now the algorithm waits for the next i -th character on the location which represents the longest suffix $t[s : i - 1]$ of $t[1 : i - 1]$ that matches a substring of $t[1 : i - 2]$, with some $2 \leq s \leq i$. Let us call this location on the path the *active point* for $i - 1$. Next the algorithm obtains the i -th character $t[i]$. If we can transit from the active point for $i - 1$ with $t[i]$, then the active point for i is the location that represents $t[s : i]$. Otherwise, the algorithm creates a new edge from the active point for $i - 1$ leading to a new leaf node, with edge label $t[i : n]$. After that, the algorithm finds the location which represents $t[s + 1 : i - 1]$ by using a suffix link, in amortized constant time. The above procedure is repeated until the active point for i is found. Readers are referred to [17] for more details of the Ukkonen algorithm.

In the sequel, we show main technical issues of our algorithm to construct $Pal_ST(t)$.

4.3.1. Suffix links

Let v be any node of $Pal_ST(t)$, and assume that the path from the root to v spells out $Lpal_w$ for some substring w of t . The *suffix link* of node v is an auxiliary edge from node v to node u , such that the path from the root to u spells out $Lpal_{w[2:|w|]}$. For example, see Fig. 3, and focus on the node which represents 1 2 1 3. The suffix link of this node points to the node which represents 1 1 3. This is because there exists a substring bbab with $Lpal_{bbab} = 1 2 1 3$, and $Lpal_{bab} = 1 1 3$.

Unlike the case of suffix trees, the node u , which is to be pointed by the suffix link of some node v , is *not* always explicit in $Pal_ST(t)$. For example, see Fig. 3. The suffix link of the node which represents 1 1 2 is illustrated to point to the implicit node which represents 1 2. In such a case, we set the suffix link of node v to the child node u' of implicit node u , and record the length of the partial edge label from u to u' . This way we can access from node v to the location for u in constant time. In the above example, the suffix link of node 1 1 2 is implemented to point to node 1 2 1 3, with auxiliary value 2 which is the length of the partial label from implicit node 1 2 to node 1 2 1 3. The same technique was used in [13] to implement the suffix links of *parameterized suffix trees*.

4.3.2. Maintaining active point

Assume that we have constructed $Pal_ST(t[1 : i - 1])$ for given string t , for some $1 \leq i \leq n$. Assume that the active point for $i - 1$ is on an implicit node u . Let v be the explicit parent node of u , and let u' be the explicit child node of v , i.e., u is on the edge from v to u' . Let x be the label of the edge from v to u' , and let ℓ be the length of the partial edge label from v to u . Then, the active point for $i - 1$, the implicit node u , is represented by $(v, x[1], s - 1 + len(v), \ell)$, where $x[1]$ is the first element of x and $s - 1$ is a position of t such that $Lpal_{t[s-1:n]}[len(v) + 1 : len(v) + \ell] = x[1 : \ell]$.

Similarly to construction of suffix trees, we look for the active point for i from the active point for $i - 1$, i.e., the implicit node u . See Fig. 5. In so doing, we use the suffix link of node v . Consider any leaf $s - 1$ in the subtree rooted at v . Let q be the node we have reached by the suffix link of node v . Now we want to look for a (possibly implicit) child y of q such that the subtree rooted at y has leaf s and $len(y) = len(u) - 1 = len(q) + \ell$. The difficulty we face is that $x[1 : \ell] = Lpal_{t[s-1:n]}[len(v) + 1 : len(v) + \ell]$ may not be equal to $Lpal_{t[s:n]}[len(q) + 1 : len(q) + \ell]$. This happens when there exists an integer k , $len(v) + 1 \leq k \leq len(v) + \ell$, such that $Lpal_{t[s-1:n]}[k] = k$. For example, see Fig. 3. The edge leading to leaf 2 is labeled with 5 1 3 3 \$, while the edge leading to leaf 3 is labeled with 2 1 3 3 \$. This is because $Lpal_{t[2:9]}[5] = 5$.

Nevertheless, we can efficiently locate y starting from q , as follows. Since $x[1] = Lpal_{t[s-1:n]}[len(v) + 1]$, we can calculate $AC_t(s - 1, s - 1 + len(v))$ in constant time. Since $len(q) = len(v) - 1$, we can compute $Lpal_{t[s:n]}[len(q) + 1 : len(q) + \ell]$ in $O(AC_t(s, s + len(q)) - AC_t(s - 1, s + len(q)) + \ell)$ time, as described in Section 3. Since there can be at most $\ell - 1$ explicit nodes in the path from q to y , we can find y in $O(\ell \cdot \min\{\sqrt{\log n}, \frac{\log \sigma}{\log \log \sigma}\})$ time, using fast integer data structures [15,16]. We check whether y is the active point for i or not, and if not, we repeat the above procedure until the active point for i is found. Since the above dynamic data structures of [15,16] permit us to insert a new element in $O(\sqrt{\log n})$ time and in amortized $O(\frac{\log \sigma}{\log \log \sigma})$ time, respectively, the total time cost after constructing $Pal_ST(t)$ is $O(n \cdot \min\{\sqrt{\log n}, \frac{\log \sigma}{\log \log \sigma}\})$. The data structures require linear space with respect to the number of integer keys to store, i.e., the number of children for each node. Hence the total space requirement for $Pal_ST(t)$ is linear in n .

Consequently, we obtain the following result.

Theorem 5. For any string t of length n , $Pal_ST(t)$ can be constructed in $O(n \cdot \min\{\sqrt{\log n}, \frac{\log \sigma}{\log \log \sigma}\})$ time with $O(n)$ space, where σ is the number of distinct characters appearing in t .

5. Conclusions and future work

Palindromes in strings have widely been studied both in theoretical and practical contexts, such as in word combinatorics and in bioinformatics. In this paper, we presented linear-time algorithms to solve a new problem called the palindrome pattern matching problem. The first algorithm is a Morris–Pratt type algorithm, and the second one is a suffix-tree type algorithm.

In practical applications such as DNA and RNA sequence analysis, it is desired to cope with *gapped palindromes* which have a spacer between the left and right arms of the palindromes. Several versions of gapped palindromes have been introduced and studied [12,18,19]. Our future work includes development of efficient solutions to a gapped-palindromes version of the palindrome pattern matching problem.

References

- [1] X. Droubay, J. Justin, G. Pirillo, Episturmian words and some constructions of de Luca and Rauzy, Theoretical Computer Science 255 (1–2) (2001) 539–553.
- [2] A. Glen, J. Justin, S. Widmer, L.Q. Zamboni, Palindromic richness, European Journal of Combinatorics 30 (2) (2009) 510–531.
- [3] M. Burrows, D.J. Wheeler, A block-sorting lossless data compression algorithm, Tech. rep., DIGITAL System Research Center (1994).
- [4] A. Restivo, G. Rosone, Burrows–Wheeler transform and palindromic richness, Theoretical Computer Science 410 (30–32) (2009) 3018–3026.
- [5] J.-P. Allouche, M. Baake, J. Cassaigne, D. Damanik, Palindrome complexity, Theoretical Computer Science 292 (1) (2003) 9–31.
- [6] S. Brlek, S. Hamel, M. Nivat, C. Reutenauer, On the palindromic complexity of infinite words, International Journal of Foundations of Computer Science 15 (2) (2004) 293–306.
- [7] M.-C. Anisuiu, V. Anisuiu, Z. Kása, Total palindrome complexity of finite words, Discrete Mathematics 310 (1) (2010) 109–114.
- [8] R. Groult, É. Prieur, G. Richomme, Counting distinct palindromes in a word in linear time, Information Processing Letters 110 (20) (2010) 908–912.
- [9] G. Manacher, A new linear-time on-line algorithm for finding the smallest initial palindrome of a string, Journal of the ACM 22 (3) (1975) 346–351.
- [10] T. I, S. Inenaga, H. Bannai, M. Takeda, Proc. SPIRE 2010, in: LNCS, vol. 6393, 2010, pp. 135–146.
- [11] A.B. Massé, S. Brlek, A. Frosini, S. Labbé, S. Rinaldi, Reconstructing words from a fixed palindromic length sequence, in: Proc. TCS 2008, in: IFIP, vol. 273, 2008, pp. 101–114.
- [12] D. Gusfield, Algorithms on Strings, Trees, and Sequences, Cambridge University Press, New York, 1997.
- [13] B.S. Baker, Parameterized pattern matching: algorithms and applications, Journal of Computer and System Sciences 52 (1) (1996) 28–42.
- [14] J.H. Morris, V.R. Pratt, A linear pattern-matching algorithm, Tech. Rep. 40, University of California, Berkeley (1970).
- [15] D.E. Willard, New trie data structures which support very fast search operations, Journal of Computer and System Sciences 28 (3) (1984) 379–394.
- [16] M.L. Fredman, D.E. Willard, Surpassing the information theoretic bound with fusion trees, Journal of Computer and System Sciences 47 (3) (1993) 424–436.
- [17] E. Ukkonen, On-line construction of suffix trees, Algorithmica 14 (3) (1995) 249–260.
- [18] R. Kolpakov, G. Kucherov, Searching for gapped palindromes, Theoretical Computer Science 410 (51) (2009) 5365–5373.
- [19] P.-H. Hsu, K.-Y. Chen, K.-M. Chao, Finding all approximate gapped palindromes, in: Proc. ISAAC 2009, in: LNCS, vol. 5878, 2009, pp. 1084–1093.